



MCP Security Audit Report

AgentSign

notebooklm-mcp-cli

18 September 2026

Report ID: MCPS-MU6M4B8D

Executive Summary

FAIL

Risk Score

100/100

10
CRITICAL

127
HIGH

0
MEDIUM

0
LOW

SCAN SCOPE 227 files 62,315 lines	LANGUAGES json (1), python (226)	MCPS SDK Not Found
---	-------------------------------------	-----------------------

Risk Comparison: WITHOUT MCPS vs WITH MCPS

CURRENT EXPOSURE			WITH MCPS PROTECTION		
MCP-01	Rug Pulls Vulnerable pattern detected, no mitigation	●	MITIGATED Passport identity binding — tools are tied to verified agent identity <code>createPassport()</code> , <code>signPassport()</code>		●
MCP-03	Privilege Escalation via Tool Composition Vulnerable pattern detected, no mitigation	●	MITIGATED Tool integrity signing — each tool definition is cryptographically signed <code>signTool()</code> , <code>verifyTool()</code>		●
MCP-04	Cross-Server Request Forgery Vulnerable pattern detected, no mitigation	●	MITIGATED Signed tool definitions prevent tampering during cross-server calls <code>signTool()</code> , <code>tool_hash</code>		●
MCP-06	Indirect Prompt Injection via MCP MCPS mitigation detected	●	MITIGATED Signed message envelopes — all MCP messages carry cryptographic signatures <code>signMessage()</code> , <code>verifyMessage()</code>		●
MCP-07	Resource Exhaustion & DoS MCPS mitigation detected	●	MITIGATED Passport verification middleware — all requests require valid signed identity <code>secureMCP()</code> , <code>verifyPassport()</code>		●
MCP-08	Insufficient Logging & Audit Vulnerable pattern detected, no mitigation	●	MITIGATED Signed audit trail — every message exchange is logged with cryptographic proof <code>auditLog()</code> , <code>onAudit()</code>		●
MCP-09	Insecure MCP-to-MCP Communication MCPS mitigation detected	●	MITIGATED Origin binding — passports encode allowed origins, verified on each request <code>validateOrigin()</code> , <code>passport.origin</code>		●
MCP-10	Context Window Pollution MCPS mitigation detected	●	MITIGATED Envelope isolation — signed message boundaries prevent context pollution <code>signMessage()</code> , <code>envelope</code>		●
Risk Reduction					
100		-->	100		

OWASP MCP Top 10 Compliance

Coverage: 4/8 risks mitigated by MCPS

ID	Risk	Status	Current State	With MCPS
MCP-01	Rug Pulls	FAIL	Vulnerable pattern detected, no mitigation	Passport identity binding — tools are tied to verified agent identity
MCP-02	Tool Poisoning	N/A	Not applicable to code-level analysis	N/A — requires LLM-level defense
MCP-03	Privilege Escalation via Tool Composition	FAIL	Vulnerable pattern detected, no mitigation	Tool integrity signing — each tool definition is cryptographically signed
MCP-04	Cross-Server Request Forgery	FAIL	Vulnerable pattern detected, no mitigation	Signed tool definitions prevent tampering during cross-server calls
MCP-05	Sampling Manipulation	N/A	Not applicable to code-level analysis	N/A — requires client-level defense
MCP-06	Indirect Prompt Injection via MCP	PASS	MCPS mitigation detected	Signed message envelopes — all MCP messages carry cryptographic signatures
MCP-07	Resource Exhaustion & DoS	PASS	MCPS mitigation detected	Passport verification middleware — all requests require valid signed identity
MCP-08	Insufficient Logging & Audit	FAIL	Vulnerable pattern detected, no mitigation	Signed audit trail — every message exchange is logged with cryptographic proof
MCP-09	Insecure MCP-to-MCP Communication	PASS	MCPS mitigation detected	Origin binding — passports encode allowed origins, verified on each request
MCP-10	Context Window Pollution	PASS	MCPS mitigation detected	Envelope isolation — signed message boundaries prevent context pollution

Quick Fix `npm install mcp-secure`

Add MCPS to your MCP server to automatically mitigate the risks marked FAIL above.

OWASP Agentic AI Top 10 Coverage

12 rules checked | 137 findings

Rule	OWASP	MITRE	STRIDE	Severity	Findings	Status
AS-001	AA-03	T1059	Elevation of Privilege	CRITICAL	7	
AS-002	AA-05	T1552	Information Disclosure	HIGH	31	
AS-003	AA-04	T1078	Elevation of Privilege	MEDIUM	0	
AS-004	AA-02	T1190	Tampering	HIGH	24	
AS-005	AA-02	T1055	Tampering	CRITICAL	3	
AS-006	AA-09	T1610	Elevation of Privilege	HIGH	1	
AS-007	AA-06	T1195	Tampering	LOW	0	
AS-008	AA-01	T1548	Elevation of Privilege	HIGH	33	
AS-009	AA-07	T1059.007	Tampering	MEDIUM	0	
AS-010	AA-08	T1562.002	Repudiation	MEDIUM	0	
AS-011	AA-10	T1041	Information Disclosure	HIGH	38	
AS-012	MCP-07	T1078.004	Spoofing	HIGH	0	

Detailed Findings

#1 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[scripts/inject_cookies_and_inspect.py:121](#)

Dangerous execution: (async function) {

```
119 |         {
120 |             "expression": ""
> 121 |         (async function() {
122 |             function sleep(ms) { return new Promise(resolve => setTimeout(resolve, ms)); }...
123 |     }
```

Remediation:

1. Replace exec()/eval() with AST-parsed execution (ast.literal_eval for data, RestrictedPython for code). 2. Run agent code in Docker/gVisor sandbox with --no-new-privileges. 3. Use allowlist of permitted modules via importlib. 4. Add mandatory human confirmation before shell commands.

#2 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[scripts/inject_cookies_and_inspect.py:172](#)

Dangerous execution: (async function) {

```
170 |         {
171 |             "expression": ""
> 172 |         (async function() {
173 |             function sleep(ms) { return new Promise(resolve => setTimeout(resolve, ms)); }
174 |     }
```

Remediation:

1. Replace exec()/eval() with AST-parsed execution (ast.literal_eval for data, RestrictedPython for code). 2. Run agent code in Docker/gVisor sandbox with --no-new-privileges. 3. Use allowlist of permitted modules via importlib. 4. Add mandatory human confirmation before shell commands.

#3 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[scripts/inspect_upload_dom.py:104](#)

Dangerous execution: (function) {

```
102 |         {
103 |             "expression": ""
> 104 |         (function() {
105 |             const walker = document.createTreeWalker(document.body, NodeFilter.SHOW_TEXT);
106 |             const matches = [];
```

Remediation:

1. Replace exec()/eval() with AST-parsed execution (ast.literal_eval for data, RestrictedPython for code). 2. Run agent code in Docker/gVisor sandbox with --no-new-privileges. 3. Use allowlist of permitted modules via importlib. 4. Add mandatory human confirmation before shell commands.

#4 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/ai_docs.py:358](#)

Unrestricted agent autonomy: - `--confirm` or `-y`: Skip confirmation (REQUIRED for automation)

```
356 |  
357 | **All generation commands support:**  
> 358 | - `--confirm` or `-y`: Skip confirmation (REQUIRED for automation)  
359 | - `--source-ids <id1,id2>`: Limit to specific sources..  
360 | - `--language <code>`: BCP-47 code, including regional locales such as `es-419`
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use `tool_choice='none'` or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#5 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/alias.py:92](#)

Unrestricted agent autonomy: `confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")`

```
90 | def delete_alias(  
91 |     name: str = typer.Argument(..., help="Alias name"),  
> 92 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),  
93 | ) -> None:  
94 |     """Delete an alias."""
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use `tool_choice='none'` or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#6 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[src/notebooklm_tools/cli/commands/doctor.py:386](#)

Dangerous execution: `result = subprocess.run( # nsec B603 — cmd from shutil.which(), args are`

```
384 |         if claude_cmd:  
385 |             try:  
> 386 |                 result = subprocess.run( # nsec B603 — cmd from shutil.which(), args are  
hardcoded strings, no shell=True  
387 |                     [claude_cmd, "mcp", "list"],  
388 |                     capture_output=True,
```

Remediation:

1. Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code). 2. Run agent code in Docker/gVisor sandbox with `--no-new-privileges`. 3. Use allowlist of permitted modules via `importlib`. 4. Add mandatory human confirmation before shell commands.

#7 AS-006 -- SANDBOXING

HIGH

OWASP: AA-09 | MITRE: T1610 | STRIDE: Elevation of Privilege

Code execution without sandboxing or isolation mechanisms

[src/notebooklm_tools/cli/commands/doctor.py:386](#)

Code execution without sandboxing

```
384 |         if claude_cmd:
385 |             try:
386 |                 result = subprocess.run( # nsec B603 - cmd from shutil.which(), args are
hardcoded strings, no shell=True
387 |                     [claude_cmd, "mcp", "list"],...
388 |                     capture_output=True,
```

Remediation:

1. Run all agent-generated code in Docker containers with --network=none and read-only filesystem. 2. Use gVisor/Firecracker for stronger isolation. 3. Set resource limits (CPU, memory, time) on execution. 4. Use E2B or Modal for managed sandboxed execution. 5. Never execute agent code in the host process.

MCPS Quick Fix: [npm install mcp-secure](#)

#8 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/label.py:102](#)

Unrestricted agent autonomy: False, "--confirm", "-y", help="Skip confirmation prompt (all-sources mode)"

```
100 |         ),
101 |         confirm: bool = typer.Option(
> 102 |             False, "--confirm", "-y", help="Skip confirmation prompt (all-sources mode)"
103 |         ),
104 |         json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#9 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/note.py:109](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
107 |         notebook_id: str = typer.Argument(..., help="Notebook ID"),
108 |         note_id: str = typer.Argument(..., help="Note ID to delete"),
> 109 |         confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
110 |         profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
111 |     ) -> None:
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#10 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/notebook.py:123](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
121 | def delete_notebook(  
122 |     notebook_id: str = typer.Argument(..., help="Notebook ID"),  
> 123 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),  
124 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...  
125 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#11 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/source.py:332](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
330 | def delete_source(  
331 |     source_ids: list[str] = typer.Argument(..., help="Source ID(s) to delete"), # noqa: B008  
> 332 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),  
333 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),  
334 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#12 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/source.py:418](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
416 |         help="Comma-separated source IDs to sync (default: all stale)",  
417 |     ),  
> 418 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),  
419 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),  
420 | ) -> None:
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#13 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:209](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
207 |     notebook_id: str = typer.Argument(..., help="Notebook ID"),
208 |     artifact_id: str = typer.Argument(..., help="Artifact ID to delete"),
> 209 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
210 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),...
211 | ) -> None:
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: npm install mcp-secure

#14 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:280](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
278 |         help="Comma-separated source IDs",
279 |     ),
> 280 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
281 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
282 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: npm install mcp-secure

#15 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:323](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
321 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
322 |     ),
> 323 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
324 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
325 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: npm install mcp-secure

#16 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:362](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
360 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
361 |     ),
> 362 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
363 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...
364 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#17 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:442](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
440 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
441 |     ),
> 442 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
443 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
444 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#18 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:472](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
470 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
471 |     ),
> 472 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
473 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
474 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#19 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:511](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
509 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
510 |     ),
> 511 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
512 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...
513 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#20 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:541](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
539 |         help='Slide revision in format: SLIDE_NUM "instruction" (e.g., --slide 1 "Make title
larger")',
540 |     ),
> 541 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
542 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
543 | ) -> None:
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#21 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:627](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
625 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
626 |     ),
> 627 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
628 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...
629 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#22 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:681](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
679 |     ),
680 |     source_ids: str | None = typer.Option(None, "--source-ids", help="Comma-separated source IDs"),
> 681 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
682 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...
683 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#23 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/studio.py:738](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
736 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
737 |     ),
> 738 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
739 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
740 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#24 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:128](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
126 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
127 |     ),
> 128 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
129 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
130 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#25 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:166](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
164 |         focus: str | None = typer.Option(None, "--focus", help="Optional focus topic"),
165 |         source_ids: str | None = typer.Option(None, "--source-ids", help="Comma-separated source IDs"),
> 166 |         confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
167 |         json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...
168 |         profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#26 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:201](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
199 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
200 |     ),
> 201 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
202 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
203 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#27 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:237](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
235 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
236 |     ),
> 237 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
238 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
239 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#28 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:268](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
266 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
267 |     ),
> 268 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
269 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...
270 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#29 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:297](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
295 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
296 |     ),
> 297 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
298 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
299 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#30 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:324](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
322 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
323 |     ),
> 324 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
325 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
326 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#31 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:348](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
346 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
347 |     ),
> 348 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
349 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...
350 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#32 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:371](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
369 |         None, "--source-ids", "-s", help="Comma-separated source IDs"
370 |     ),
> 371 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
372 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
373 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#33 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:519](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
517 | def delete_notebook_verb(
518 |     notebook: str = typer.Argument(..., help="Notebook ID or alias"),
> 519 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),
520 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),
521 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#34 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:535](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
533 | def delete_source_verb(  
534 |     source: str = typer.Argument(..., help="Source ID"),  
> 535 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),  
536 |     json_output: bool = typer.Option(False, "--json", "-j", help="Output as JSON"),...  
537 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#35 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:552](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
550 |     notebook: str = typer.Argument(..., help="Notebook ID or alias"),  
551 |     artifact: str = typer.Argument(..., help="Artifact ID"),  
> 552 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),  
553 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),  
554 | ) -> None:
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#36 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:562](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation p

```
560 | def delete_alias_verb(  
561 |     name: str = typer.Argument(..., help="Alias name"),  
> 562 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation prompt"),  
563 | ) -> None:  
564 |     """Delete an alias."""
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use tool_choice='none' or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: [npm install mcp-secure](#)

#37 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/commands/verbs.py:820](#)

Unrestricted agent autonomy: confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation")

```
818 |         None, "--source-ids", "-s", help="Comma-separated source IDs to sync (default: all stale)"
819 |     ),
> 820 |     confirm: bool = typer.Option(False, "--confirm", "-y", help="Skip confirmation"),
821 |     profile: str | None = typer.Option(None, "--profile", "-p", help="Profile to use"),...
822 | ) -> None:
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use `tool_choice='none'` or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#38 AS-008 -- EXCESSIVE AGENCY

HIGH

OWASP: AA-01 | MITRE: T1548 | STRIDE: Elevation of Privilege

Agent configured with unrestricted tool access or auto-approval of dangerous actions

[src/notebooklm_tools/cli/main.py:701](#)

Unrestricted agent autonomy: help="Skip confirmation prompt",

```
699 |         "--confirm",
700 |         "-y",
> 701 |         help="Skip confirmation prompt",
702 |     ),
703 | ) -> None:
```

Remediation:

1. Implement mandatory human-in-the-loop for destructive operations (delete, write, execute, send). 2. Use `tool_choice='none'` or explicit tool allowlists instead of 'auto'. 3. Add confirmation prompts before irreversible actions. 4. Log all tool invocations with full parameters for audit.

MCPS Quick Fix: `npm install mcp-secure`

#39 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/cli/utils.py:149](#)

File input near prompt/instruction handling — injection vector

```
147 |
148 |     try:
> 149 |         with open(cache_path, encoding="utf-8") as f:
150 |             data = json.load(f)
151 |
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#40 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/cli/utils.py:167](#)

File input near prompt/instruction handling — injection vector

```
165 |         return
166 |     try:
> 167 |         with open(cache_path, "w", encoding="utf-8") as f:
168 |             json.dump(...)
169 |         {
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#41 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/cli/utils.py:184](#)

File input near prompt/instruction handling — injection vector

```
182 |         url = "https://pypi.org/pypi/notebooklm-mcp-cli/json"
183 |         req = urllib.request.Request(url, headers={"User-Agent": "notebooklm-mcp-cli"})
> 184 |         with urllib.request.urlopen(req, timeout=2) as response: # nosec B310 – URL is hardcoded to
https://pypi.org
185 |             data = json.loads(response.read().decode())
186 |             return data.get("info", {}).get("version")
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#42 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/cli/utils.py:6](#)

Dynamic HTTP with sensitive data context

```
4 | import sys
5 | import time
> 6 | import urllib.request
7 | from pathlib import Path
8 |
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#43 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/cli/utils.py:183](#)

Dynamic HTTP with sensitive data context

```
181 |         try:
182 |             url = "https://pypi.org/pypi/notebooklm-mcp-cli/json"
> 183 |             req = urllib.request.Request(url, headers={"User-Agent": "notebooklm-mcp-cli"})
184 |             with urllib.request.urlopen(req, timeout=2) as response: # nosec B310 – URL is hardcoded to
https://pypi.org
185 |                 data = json.loads(response.read().decode())
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#44 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/cli/utlis.py:184](#)

Dynamic HTTP with sensitive data context

```
182 |         url = "https://pypi.org/pypi/notebooklm-mcp-cli/json"
183 |         req = urllib.request.Request(url, headers={"User-Agent": "notebooklm-mcp-cli"})
> 184 |         with urllib.request.urlopen(req, timeout=2) as response: # nsec B310 - URL is hardcoded to
https://pypi.org
185 |             data = json.loads(response.read().decode())...
186 |             return data.get("info", {}).get("version")
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#45 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/auth.py:133](#)

File input near prompt/instruction handling — injection vector

```
131 |
132 |     try:
> 133 |         with open(cache_path, encoding="utf-8") as f:
134 |             data = json.load(f)
135 |             tokens = AuthTokens.from_dict(data)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#46 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/auth.py:177](#)

File input near prompt/instruction handling — injection vector

```
175 |         cache_path.parent.chmod(stat.S_IRWXU) # 0o700
176 |         # TOCTOU-safe: create the file with 0o600 from the start (PR #205)
> 177 |         fd = os.open(str(cache_path), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
178 |         try:
179 |             f = os.fdopen(fd, "w", encoding="utf-8")
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#47 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/auth.py:179](#)

File input near prompt/instruction handling — injection vector

```
177 |         fd = os.open(str(cache_path), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
178 |         try:
> 179 |             f = os.fdopen(fd, "w", encoding="utf-8")
180 |         except BaseException:
181 |             os.close(fd)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#48 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/auth.py:505](#)

File input near prompt/instruction handling — injection vector

```
503 |
504 |         # Save cookies with restrictive permissions from creation
> 505 |         fd = os.open(str(self.cookies_file), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
506 |         try:
507 |             f = os.fdopen(fd, "w", encoding="utf-8")
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#49 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/auth.py:507](#)

File input near prompt/instruction handling — injection vector

```
505 |         fd = os.open(str(self.cookies_file), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
506 |         try:
> 507 |             f = os.fdopen(fd, "w", encoding="utf-8")
508 |         except BaseException:
509 |             os.close(fd)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#50 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/auth.py:534](#)

File input near prompt/instruction handling — injection vector

```
532 |         "last_validated": datetime.now().isoformat(),
533 |     }
> 534 |     fd = os.open(str(self.metadata_file), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
535 |     try:
536 |         f = os.fdopen(fd, "w", encoding="utf-8")
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#51 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/auth.py:536](#)

File input near prompt/instruction handling — injection vector

```
534 |         fd = os.open(str(self.metadata_file), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
535 |         try:
536 |             f = os.fdopen(fd, "w", encoding="utf-8")
537 |         except BaseException:
538 |             os.close(fd)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#52 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/auth.py:17](#)

Dynamic HTTP with sensitive data context

```
15 | from pathlib import Path
16 | from typing import Any
> 17 | from urllib.parse import quote
18 |
19 | from notebooklm_tools.utils.config import get_base_url
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#53 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/auth.py:614](#)

Dynamic HTTP with sensitive data context

```
612 |     def login_with_file(self, file_path: str | Path) -> Profile:
613 |         """Parse cookies from file and save to profile."""
> 614 |         from urllib.parse import urlparse
615 |
616 |         from notebooklm_tools.core.exceptions import AuthenticationError
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#54 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/base.py:1274](#)

File input near prompt/instruction handling — injection vector

```
1272 |         # writing first and narrowing after, which leaves a window in
1273 |         # which the file is world-readable.
> 1274 |         fd = os.open(str(debug_path), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
1275 |         try:...
1276 |             f = os.fdopen(fd, "w", encoding="utf-8")
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#55 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/base.py:1276](#)

File input near prompt/instruction handling — injection vector

```
1274 |         fd = os.open(str(debug_path), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
1275 |         try:
> 1276 |             f = os.fdopen(fd, "w", encoding="utf-8")
1277 |         except BaseException:
1278 |             os.close(fd)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#56 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/base.py:18](#)

Dynamic HTTP with sensitive data context

```
16 | import secrets
17 | import threading
> 18 | import urllib.parse
19 | from collections import OrderedDict
20 | from typing import Any
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#57 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/base.py:690](#)

Dynamic HTTP with sensitive data context

```
688 |
689 |         # URL encode (safe='' encodes all characters including /)
> 690 |         body_parts = [f"f.req={urllib.parse.quote(f_req_json, safe='')}"]
691 |
692 |         if self.csrf_token:
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#58 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/base.py:693](#)

Dynamic HTTP with sensitive data context

```
691 |  
692 |         if self.csrf_token:  
> 693 |             body_parts.append(f"at={urllib.parse.quote(self.csrf_token, safe='')}")  
694 | ...  
695 |         # Add trailing & to match NotebookLM's format
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#59 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/base.py:712](#)

Dynamic HTTP with sensitive data context

```
710 |         params["f.sid"] = self._session_id  
711 |  
> 712 |         query = urllib.parse.urlencode(params)  
713 |         return f"{self._get_batchexecute_url()}?{query}"  
714 |
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#60 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/conversation.py:12](#)

Dynamic HTTP with sensitive data context

```
10 | import os  
11 | import time  
> 12 | import urllib.parse  
13 | from typing import Any, Protocol, cast  
14 |
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#61 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/conversation.py:504](#)

Dynamic HTTP with sensitive data context

```
502 |  
503 | # URL encode with safe='' to encode all characters including /  
> 504 | body_parts = [f"f.req={urllib.parse.quote(f_req_json, safe='')}"]  
505 | if self.csrf_token:...  
506 |     body_parts.append(f"at={urllib.parse.quote(self.csrf_token, safe='')}")
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#62 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/conversation.py:506](#)

Dynamic HTTP with sensitive data context

```
504 | body_parts = [f"f.req={urllib.parse.quote(f_req_json, safe='')}"]  
505 | if self.csrf_token:  
> 506 |     body_parts.append(f"at={urllib.parse.quote(self.csrf_token, safe='')}")  
507 | # Add trailing & to match NotebookLM's format  
508 | body = "&".join(body_parts) + "&"
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#63 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/conversation.py:527](#)

Dynamic HTTP with sensitive data context

```
525 | url_params["f.sid"] = self._session_id  
526 |  
> 527 | query_string = urllib.parse.urlencode(url_params)  
528 | endpoint = getattr(self, "_get_query_endpoint", lambda: self.QUERY_ENDPOINT)()  
529 | url = f"{self._get_base_url()}{endpoint}?{query_string}"
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#64 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/download.py:150](#)

File input near prompt/instruction handling — injection vector

```
148 |         {"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36"},  
149 |     )  
> 150 |     # Mirror Chrome's window.open() from notebooklm.google.com to a cross-domain  
151 |     # artifact host (lh3.googleusercontent.com / lh3.google.com). The audio CDN  
152 |     # treats "Sec-Fetch-Site: none" as an address-bar navigation and returns 403;
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#65 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/download.py:153](#)

File input near prompt/instruction handling — injection vector

```
151 |         # artifact host (lh3.googleusercontent.com / lh3.google.com). The audio CDN
> 152 |         # treats "Sec-Fetch-Site: none" as an address-bar navigation and returns 403;
153 |         # the UI's Download button succeeds because window.open() makes Chrome stamp
154 |         # the request with "Sec-Fetch-Site: cross-site" + Referer=notebooklm.google.com....
155 |         headers = {
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#66 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/download.py:211](#)

File input near prompt/instruction handling — injection vector

```
209 |
> 210 |         # Not an auth error - write first chunk and continue
211 |         with open(temp_file, "wb") as f:
212 |             f.write(first_chunk)
213 |             bytes_downloaded = len(first_chunk)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#67 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/download.py:228](#)

File input near prompt/instruction handling — injection vector

```
226 |         # Binary content - stream directly
227 |         bytes_downloaded = 0
> 228 |         with open(temp_file, "wb") as f:
229 |             async for chunk in response.aiter_bytes(chunk_size=chunk_size):
230 |                 f.write(chunk)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#68 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/download.py:288](#)

File input near prompt/instruction handling — injection vector

```
286 |         ):
287 |             response.raise_for_status()
> 288 |             with open(temp_file, "wb") as output:
289 |                 for chunk in response.iter_bytes(chunk_size=65536):...
290 |                     output.write(chunk)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#69 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/core/download.py:1086](#)

File input near prompt/instruction handling — injection vector

```
1084 |         output.parent.mkdir(parents=True, exist_ok=True)
1085 |
> 1086 |         with open(output, "w", newline="", encoding="utf-8-sig") as f:
1087 |             writer = csv.writer(f)
1088 |             writer.writerow(headers)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#70 AS-005 -- INJECTION PATTERN

CRITICAL

OWASP: AA-02 | MITRE: T1055 | STRIDE: Tampering

Known prompt injection, SQL injection, XSS, or command injection patterns

[src/notebooklm_tools/core/download.py:1285](#)

Known injection pattern detected

```
1283 |         Tries multiple extraction patterns for robustness:
1284 |         1. data-app-data attribute (primary)
> 1285 |         2. <script id="application-data"> tag (fallback)
1286 |         3. Other common patterns
1287 |
```

Remediation:

1. Validate and sanitize all inputs with strict allowlists. 2. Use parameterized queries for any database access (never string concatenation). 3. HTML-encode all agent output before rendering. 4. Implement content security policy (CSP) headers. 5. Block known injection patterns at the input boundary.

#71 AS-005 -- INJECTION PATTERN

CRITICAL

OWASP: AA-02 | MITRE: T1055 | STRIDE: Tampering

Known prompt injection, SQL injection, XSS, or command injection patterns

[src/notebooklm_tools/core/download.py:1313](#)

Known injection pattern detected

```
1311 |         logger.debug(f"JSON preview: {decoded_json[:200]}...")
1312 |
> 1313 |         # Pattern 2: <script id="application-data"> tag
1314 |         match = re.search(
1315 |             r'<script[^\>]+id=["\']application-data["\'][^\>]*>(.*?)</script>',
```

Remediation:

1. Validate and sanitize all inputs with strict allowlists. 2. Use parameterized queries for any database access (never string concatenation). 3. HTML-encode all agent output before rendering. 4. Implement content security policy (CSP) headers. 5. Block known injection patterns at the input boundary.

#72 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/download.py:12](#)

Dynamic HTTP with sensitive data context

```
10 | from pathlib import Path
11 | from typing import Any
> 12 | from urllib.parse import urlparse
13 | ...
14 | import httpx
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#73 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/utils.py:4](#)

Dynamic HTTP with sensitive data context

```
2 |
3 | import json
> 4 | import urllib.parse
5 | from datetime import UTC, datetime
6 | from typing import Any
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#74 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/utils.py:68](#)

Dynamic HTTP with sensitive data context

```
66 |     result = {}
67 |     try:
> 68 |         parsed = urllib.parse.parse_qs(body.rstrip("&"))
69 |         if "f.req" in parsed:
70 |             f_req_raw = parsed["f.req"][0]
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#75 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/utils.py:98](#)

Dynamic HTTP with sensitive data context

```
96 |     """Parse URL query parameters for debug display."""
97 |     try:
> 98 |         parsed = urllib.parse.urlparse(url)
99 |         params = urllib.parse.parse_qs(parsed.query)...
100 |         return {k: v[0] if len(v) == 1 else v for k, v in params.items()}
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#76 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/core/utils.py:99](#)

Dynamic HTTP with sensitive data context

```
97 |     try:
98 |         parsed = urllib.parse.urlparse(url)
> 99 |         params = urllib.parse.parse_qs(parsed.query)
100 |         return {k: v[0] if len(v) == 1 else v for k, v in params.items()}
101 |     except Exception:
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#77 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/mcp/tools/auth.py:5](#)

Dynamic HTTP with sensitive data context

```
3 | import os
4 | import time
> 5 | import urllib.parse
6 | from http.cookies import SimpleCookie
7 |
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#78 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/mcp/tools/auth.py:155](#)

Dynamic HTTP with sensitive data context

```
153 |         # Try to extract CSRF token from request body if provided
154 |         if not csrf_token and request_body:
> 155 |             body_params = urllib.parse.parse_qs(request_body, keep_blank_values=True)
156 |             csrf_token = body_params.get("at", [""])[0]
157 |
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#79 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/mcp/tools/auth.py:161](#)

Dynamic HTTP with sensitive data context

```
159 |         build_label = ""
160 |         if request_url:
> 161 |             parsed_url = urllib.parse.urlparse(request_url)
162 |             query = parsed_url.query or request_url...
163 |             url_params = urllib.parse.parse_qs(query, keep_blank_values=True)
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#80 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/mcp/tools/auth.py:163](#)

Dynamic HTTP with sensitive data context

```
161 |         parsed_url = urllib.parse.urlparse(request_url)
162 |         query = parsed_url.query or request_url
> 163 |         url_params = urllib.parse.parse_qs(query, keep_blank_values=True)
164 |         if not session_id:
165 |             session_id = url_params.get("f.sid", [""])[0]
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#81 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/mcp/tools/server.py:4](#)

Dynamic HTTP with sensitive data context

```
2 |
3 | import json
> 4 | import urllib.request
5 | from typing import Any, cast
6 |
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#82 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/mcp/tools/server.py:20](#)

Dynamic HTTP with sensitive data context

```
18 |         try:
19 |             url = "https://pypi.org/pypi/notebooklm-mcp-cli/json"
> 20 |             req = urllib.request.Request(url, headers={"User-Agent": "notebooklm-mcp-cli"})
21 |             with urllib.request.urlopen(req, timeout=2) as response: # nosec B310 - URL is hardcoded to
https://pypi.org...
22 |                 data = cast(dict[str, Any], json.loads(response.read().decode()))
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#83 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/mcp/tools/server.py:21](#)

Dynamic HTTP with sensitive data context

```
19 |         url = "https://pypi.org/pypi/notebooklm-mcp-cli/json"
20 |         req = urllib.request.Request(url, headers={"User-Agent": "notebooklm-mcp-cli"})
> 21 |         with urllib.request.urlopen(req, timeout=2) as response: # nosec B310 - URL is hardcoded to
https://pypi.org
22 |             data = cast(dict[str, Any], json.loads(response.read().decode()))
23 |             info = data.get("info")
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#84 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/services/auth_replay.py:245](#)

Dynamic HTTP with sensitive data context

```
243 |         return _fail("NotebookLM page loaded, but session fields were not found.")
244 |
> 245 |         from urllib.parse import urlparse
246 |
247 |         csrf_token = cdp.extract_csrf_token(html) or profile.csrf_token or ""
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#85 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/services/sources.py:5](#)

Dynamic HTTP with sensitive data context

```
3 | import os
4 | import time
> 5 | import urllib.parse
6 | from concurrent.futures import ThreadPoolExecutor...
7 | from pathlib import Path
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#86 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/services/sources.py:178](#)

Dynamic HTTP with sensitive data context

```
176 |         if not url:
177 |             raise ValidationError("url is required for source_type='url'")
> 178 |         parsed = urllib.parse.urlparse(url)
179 |         if not parsed.scheme or parsed.scheme.lower() not in ALLOWED_URL_SCHEMES:
180 |             raise ValidationError()
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#87 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/utils/cdp.py:248](#)

File input near prompt/instruction handling — injection vector

```
246 |     map_file = _get_port_map_file()
247 |     try:
> 248 |         fd = os.open(str(map_file), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
249 |         try:
250 |             f = os.fdopen(fd, "w", encoding="utf-8")
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#88 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/utils/cdp.py:250](#)

File input near prompt/instruction handling — injection vector

```
248 |         fd = os.open(str(map_file), os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
249 |         try:
> 250 |             f = os.fdopen(fd, "w", encoding="utf-8")
251 |         except BaseException:
252 |             os.close(fd)
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#89 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/utils/cdp.py:968](#)

File input near prompt/instruction handling — injection vector

```
966 |         try:
967 |             _logger.debug("Launching browser: %s on port %d", chrome_path, port)
> 968 |             process = subprocess.Popen(args, **kwargs) # nsec B603 – chrome_path from shutil.which();
args built from hardcoded flags + validated port
969 |             return process...
970 |         except Exception as e:
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#90 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/utils/cdp.py:23](#)

Dynamic HTTP with sensitive data context

```
21 | from pathlib import Path
22 | from typing import Any
> 23 | from urllib.parse import quote, urlparse
24 |
25 | from httpx import Client, HTTPTransport
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#91 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/utils/config.py:14](#)

Dynamic HTTP with sensitive data context

```
12 | from pathlib import Path
13 | from typing import Any
> 14 | from urllib.parse import quote, urlparse
15 |
16 | from pydantic import BaseModel, Field
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#92 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/utils/config.py:90](#)

Dynamic HTTP with sensitive data context

```
88 |         export NOTEBOOKLM_BASE_URL=https://notebooklm.cloud.google.com
89 |         """
> 90 |         from urllib.parse import urlparse
91 |     ...
92 |     env_override = os.environ.get("NOTEBOOKLM_BASE_URL")
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#93 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/utils/firefox.py:102](#)

File input near prompt/instruction handling — injection vector

```
100 | def _launch_firefox(firefox_path: str, profile_dir: Path) -> subprocess.Popen[bytes]:
101 |     try:
> 102 |         return subprocess.Popen(
103 |             [
104 |                 firefox_path,
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#94 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[src/notebooklm_tools/utils/wsl.py:338](#)

File input near prompt/instruction handling — injection vector

```
336 |         logger.debug(f"Chrome temp profile: {temp_dir}")
337 |         try:
> 338 |             process = subprocess.Popen( # nsec B603 B607 — args built from validated Chrome path +
hardcoded flags + int port
339 |                 args,
340 |                 stdout=stdout_arg,
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

#95 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/utils/wsl.py:365](#)

Dynamic HTTP with sensitive data context

```
363 |         """ True if CDP is ready, False if timeout.
364 |         """
> 365 |         import urllib.parse
366 |
367 |         parsed = urllib.parse.urlparse(cdp_url)
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#96 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[src/notebooklm_tools/utils/wsl.py:367](#)

Dynamic HTTP with sensitive data context

```
365 | import urllib.parse
366 |
> 367 | parsed = urllib.parse.urlparse(cdp_url)
368 | base_url = f"{parsed.scheme}://{parsed.hostname}:{parsed.port}"...
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#97 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/cli/test_usage_cli.py:74](#)

Potential secret: possible hardcoded secret

```
72 | from notebooklm_tools.services.auth import AuthManager
73 |
> 74 | AuthManager("work").save_profile(cookies={"SID": "work"}, csrf_token="work-csrf")
75 | monkeypatch.setenv("NOTEBOOKLM_COOKIES", "SID=personal")
76 | with (
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#98 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_auth_check.py:101](#)

Potential secret: possible hardcoded secret

```
99 |         "SAPISID": "dead",
100 |     },
> 101 |     csrf_token="stale123",
102 |     session_id="sess123",
103 |     email="test@example.com",
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#99 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_auth_check.py:141](#)

Potential secret: possible hardcoded secret

```
139 |         "SAPISID": "dead",
140 |     },
> 141 |     csrf_token="stale123",
142 |     session_id="sess123",...
143 |     email="test@example.com",
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#100 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_auth_recovery.py:26](#)

Potential secret: possible hardcoded secret

```
24 | def _make_client(cookies):
25 |     with patch.object(NotebookLMClient, "_refresh_auth_tokens"):
> 26 |         return NotebookLMClient(cookies=cookies, csrf_token="stale_token")
27 |
28 |
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#101 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_auth_recovery.py:42](#)

Potential secret: possible hardcoded secret

```
40 |         if client.cookies == {"SID": "stale"}:
41 |             raise ValueError("Authentication expired. accounts.google.com login")
> 42 |         client.csrf_token = "fresh_token"
43 |
44 |         # Disk holds different (fresher) cookies than what is in memory.
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#102 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_base.py:34](#)

Potential secret: possible hardcoded secret

```
32 |     # When csrf_token is provided, _refresh_auth_tokens should NOT be called
33 |     with patch.object(BaseClient, "_refresh_auth_tokens") as mock_refresh:
> 34 |         client = BaseClient(cookies={"test": "cookie"}, csrf_token="test_token")
35 |         mock_refresh.assert_not_called()
36 |         assert client.csrf_token == "test_token"
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#103 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_base.py:73](#)

Potential secret: possible hardcoded secret

```
71 |  
> 72 |         with patch.object(BaseClient, "_refresh_auth_tokens"):  
73 |             client = BaseClient(cookies={}, csrf_token="test_token")  
74 |             body = client._build_request_body("testRpc", ["param1"])...  
75 |             assert "f.req=" in body
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#104 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_base.py:85](#)

Potential secret: possible hardcoded secret

```
83 |  
> 84 |         with patch.object(BaseClient, "_refresh_auth_tokens"):  
85 |             client = BaseClient(cookies={}, csrf_token="test_token", session_id="test_sid")  
86 |             url = client._build_url("testRpc", "/notebook/123")  
87 |             assert "rpcids=testRpc" in url
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#105 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_base.py:247](#)

Potential secret: possible hardcoded secret

```
245 |         cached = AuthTokens(  
246 |             cookies={"SID": "old"},  
> 247 |             csrf_token="old-csrf",  
248 |             session_id="old-session",  
249 |         )
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#106 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_base.py:258](#)

Potential secret: possible hardcoded secret

```
256 |         client = BaseClient(  
257 |             cookies={"SID": "rotated"},  
> 258 |             csrf_token="new-csrf",  
259 |             session_id="new-session", ...  
260 |         )
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#107 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:24](#)

Potential secret: possible hardcoded secret

```
22 |         client = BaseClient(  
23 |             cookies={"SID": "old"},  
> 24 |             csrf_token="old-csrf",  
25 |             session_id="old-session",  
26 |             build_label="old-build",
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#108 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:34](#)

Potential secret: possible hardcoded secret

```
32 |     def prepare_cdp_transport(self, timeout):  
33 |         assert timeout == 45.0  
> 34 |         self.csrf_token = "live-csrf"  
35 |         self._session_id = "live-session"  
36 |         self._bl = "live-build"
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#109 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:71](#)

Potential secret: possible hardcoded secret

```
69 |         client = BaseClient(  
70 |             cookies={"SID": "old"},  
> 71 |             csrf_token="old-csrf",  
72 |             session_id="old-session",  
73 |             build_label="old-build",
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#110 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:84](#)

Potential secret: possible hardcoded secret

```
82 |         port=9222,  
> 83 |         ws_url="ws://page",  
84 |         csrf_token="live-csrf",  
85 |         session_id="live-session", ...  
86 |         build_label="live-build",
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#111 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:109](#)

Potential secret: possible hardcoded secret

```
107 |         mixin = ConversationMixin(  
> 108 |             cookies={"SID": "old"},  
109 |             csrf_token="old-csrf",  
110 |             session_id="old-session",  
111 |             build_label="old-build",
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#112 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:116](#)

Potential secret: possible hardcoded secret

```
114 |         def prepare_cdp_transport(self, timeout):  
> 115 |             assert 0 < timeout <= 99.0  
116 |             self.csrf_token = "live-csrf"  
117 |             self._session_id = "live-session"  
118 |             self._bl = "live-build"
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#113 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:206](#)

Potential secret: possible hardcoded secret

```
204 |  
> 205 |     with patch.object(BaseClient, "_refresh_auth_tokens"):  
206 |         client = BaseClient(cookies={"SID": "old"}, csrf_token="old-csrf")  
207 | ...  
208 |     def fake_get_context(**kwargs):
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#114 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_cdp_transport.py:213](#)

Potential secret: possible hardcoded secret

```
211 |  
> 212 |         port=9230,  
213 |         ws_url="ws://page",  
214 |         csrf_token="live-csrf",  
215 |         session_id="live-session",  
        build_label="live-build",
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#115 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[tests/core/test_cdp_transport.py:4](#)

Dynamic HTTP with sensitive data context

```
2 |  
> 3 | import json  
4 | import urllib.parse  
5 | from unittest.mock import patch  
6 |
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#116 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[tests/core/test_cdp_transport.py:53](#)

Dynamic HTTP with sensitive data context

```
51 |  
> 52 |     assert result == ["ok"]  
53 |     query_params = urllib.parse.parse_qs(urllib.parse.urlparse(captured["url"]).query)  
54 |     assert query_params["f.sid"] == ["live-session"]  
55 |     assert query_params["bl"] == ["live-build"]
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#117 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[tests/core/test_cdp_transport.py:56](#)

Dynamic HTTP with sensitive data context

```
54 |     assert query_params["f.sid"] == ["live-session"]
55 |     assert query_params["bl"] == ["live-build"]
> 56 |     body_params = urllib.parse.parse_qs(captured["body"])
57 |     assert body_params["at"] == ["live-csrf"]...
58 |     assert captured["timeout"] == 45.0
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#118 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[tests/core/test_cdp_transport.py:155](#)

Dynamic HTTP with sensitive data context

```
153 |     assert result["conversation_id"] == "server-conv"
154 |     assert "GenerateFreeFormStreamed" in str(captured["url"])
> 155 |     query_params = urllib.parse.parse_qs(urllib.parse.urlparse(str(captured["url"])).query)
156 |     assert query_params["f.sid"] == ["live-session"]
157 |     assert query_params["bl"] == ["live-build"]
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#119 AS-011 -- DATA EXFILTRATION

HIGH

OWASP: AA-10 | MITRE: T1041 | STRIDE: Information Disclosure

Dynamic HTTP requests that could exfiltrate sensitive data to attacker-controlled endpoints

[tests/core/test_cdp_transport.py:158](#)

Dynamic HTTP with sensitive data context

```
156 |     assert query_params["f.sid"] == ["live-session"]
157 |     assert query_params["bl"] == ["live-build"]
> 158 |     body_params = urllib.parse.parse_qs(str(captured["body"]))
159 |     assert body_params["at"] == ["live-csrf"]
160 |     assert 0 < captured["timeout"] <= 99.0
```

Remediation:

1. Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains. 2. Use egress firewalls or network policies to restrict outbound traffic. 3. Redact sensitive data (tokens, credentials) before passing to agent context. 4. Log all outbound HTTP requests with destination and payload size. 5. Block requests containing sensitive patterns in the body.

#120 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_socks_proxy.py:33](#)

Potential secret: possible hardcoded secret

```
31 | def test_sync_client_initializes_with_socks_proxy_env(monkeypatch: pytest.MonkeyPatch) -> None:
32 |     _configure_socks_proxy(monkeypatch)
> 33 |     client = BaseClient(cookies={}, csrf_token="test-token")
34 | ...
35 |     http_client = client._get_client()
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#121 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_socks_proxy.py:46](#)

Potential secret: possible hardcoded secret

```
44 | ) -> None:
45 |     _configure_socks_proxy(monkeypatch)
> 46 |     client = BaseClient(cookies={}, csrf_token="test-token")
47 |
48 |     http_client = client._get_async_client()
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#122 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/core/test_thread_safety.py:19](#)

Potential secret: possible hardcoded secret

```
17 |
18 |     with patch.object(BaseClient, "_refresh_auth_tokens"):
> 19 |         return NotebookLMClient(cookies={"test": "cookie"}, csrf_token="test_token")
20 |
21 |
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#123 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[tests/services/test_auth_health.py:340](#)

Dangerous execution: usable, status, detail = __import__(

```
338 |         lambda: type("C", (), {"check": lambda self, **kw: report})(),
339 |         )
> 340 |         usable, status, detail = __import__(
341 |             "notebooklm_tools.services.auth", fromlist=["credentials_are_usable"]
342 |         ).credentials_are_usable()
```

Remediation:

1. Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code). 2. Run agent code in Docker/gVisor sandbox with `--no-new-privileges`. 3. Use allowlist of permitted modules via `importlib`. 4. Add mandatory human confirmation before shell commands.

#124 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[tests/services/test_auth_service.py:171](#)

Dangerous execution: `exec()` (# nsec B102 - fixed literal import statement, no user input

```
169 |         # Re-execute the import statement the same way cli code does.
170 |         local_namespace = {}
> 171 |         exec( # nsec B102 - fixed literal import statement, no user input
172 |             "from notebooklm_tools.services.auth import AuthManager", local_namespace...
173 |         )
```

Remediation:

1. Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code). 2. Run agent code in Docker/gVisor sandbox with `--no-new-privileges`. 3. Use allowlist of permitted modules via `importlib`. 4. Add mandatory human confirmation before shell commands.

#125 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_api_client.py:15](#)

Potential secret: possible hardcoded secret

```
13 |     cookies = {"SID": "test_sid"}
14 |     with patch.object(NotebookLMClient, "_refresh_auth_tokens"):
> 15 |         client = NotebookLMClient(cookies=cookies, csrf_token="old_token", session_id="old_sid")
16 |         return client
17 |
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add `.env` to `.gitignore`.

#126 AS-005 -- INJECTION PATTERN

CRITICAL

OWASP: AA-02 | MITRE: T1055 | STRIDE: Tampering

Known prompt injection, SQL injection, XSS, or command injection patterns

[tests/test_api_client.py:166](#)

Known injection pattern detected

```
164 |         # Simplified HTML to minimize whitespace issues
165 |         # api_client.py looks for "FdrFJe":"..." for the session ID
> 166 |         html = '<html><script>WIZ_global_data = {"SNLM0e":"new_csrf_token", "FdrFJe":"123456789"};</
script></html>'
167 |
168 |         # Restore the original method for this test
```

Remediation:

1. Validate and sanitize all inputs with strict allowlists. 2. Use parameterized queries for any database access (never string concatenation). 3. HTML-encode all agent output before rendering. 4. Implement content security policy (CSP) headers. 5. Block known injection patterns at the input boundary.

#127 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_file_upload.py:99](#)

Potential secret: possible hardcoded secret

```
97 |         client = SourceMixin.__new__(SourceMixin)
98 |         client.cookies = {"test": "cookie"}
> 99 |         client.csrf_token = "test-csrf"
100 |         client._session_id = "test-session"...
101 |         client._client = None
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#128 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_file_upload.py:124](#)

Potential secret: possible hardcoded secret

```
122 |         client = SourceMixin.__new__(SourceMixin)
123 |         client.cookies = {"test": "cookie"}
> 124 |         client.csrf_token = "test-csrf"
125 |         client._session_id = "test-session"
126 |         client._client = None
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#129 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_file_upload.py:147](#)

Potential secret: possible hardcoded secret

```
145 |         client = SourceMixin.__new__(SourceMixin)
146 |         client.cookies = {"test": "cookie"}
> 147 |         client.csrf_token = "test-csrf"
148 |         client._session_id = "test-session"
149 |         client._client = None
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#130 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_file_upload.py:178](#)

Potential secret: possible hardcoded secret

```
176 |         client = SourceMixin.__new__(SourceMixin)
177 |         client.cookies = {"test": "cookie"}
> 178 |         client.csrf_token = "test-csrf"
179 |         client._session_id = "test-session"
180 |         client._client = None
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#131 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_file_upload.py:208](#)

Potential secret: possible hardcoded secret

```
206 |         client = SourceMixin.__new__(SourceMixin)
> 207 |         client.cookies = {"test": "cookie"}
    208 |         client.csrf_token = "test-csrf"
    209 |         client._session_id = "test-session"...
    210 |         client._client = None
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#132 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_file_upload.py:248](#)

Potential secret: possible hardcoded secret

```
246 |         client = SourceMixin.__new__(SourceMixin)
> 247 |         client.cookies = {"test": "cookie"}
    248 |         client.csrf_token = "test-csrf"
    249 |         client._session_id = "test-session"
    250 |         client._client = None
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#133 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_file_upload.py:287](#)

Potential secret: possible hardcoded secret

```
285 |         client = SourceMixin.__new__(SourceMixin)
> 286 |         client.cookies = {"test": "cookie"}
    287 |         client.csrf_token = "test-csrf"
    288 |         client._session_id = "test-session"
    289 |         client._client = None
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (detect-secrets, trufflehog) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add .env to .gitignore.

#134 AS-001 -- UNSAFE EXECUTION

CRITICAL

OWASP: AA-03 | MITRE: T1059 | STRIDE: Elevation of Privilege

Dangerous code execution patterns that allow arbitrary command injection

[tests/test_io_encoding_windows.py:37](#)

Dangerous execution: with `__import__`("pytest").raises(UnicodeEncodeError):

```
35 |  
36 |     stream = io.TextIOWrapper(io.BytesIO(), encoding="cp1252", errors="strict")  
> 37 |     with __import__("pytest").raises(UnicodeEncodeError):  
38 |         stream.write("\u2192")...  
39 |
```

Remediation:

1. Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code). 2. Run agent code in Docker/gVisor sandbox with `--no-new-privileges`. 3. Use allowlist of permitted modules via `importlib`. 4. Add mandatory human confirmation before shell commands.

#135 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_profile_cache_isolation.py:37](#)

Potential secret: possible hardcoded secret

```
35 |     client = get_client("tsm")  
36 |     client.cookies = {"SID": "account-B-refreshed"}  
> 37 |     client.csrf_token = "csrf-b-refreshed"  
38 |     client._session_id = "session-b-refreshed"  
39 |     client._update_cached_tokens()
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (`detect-secrets`, `trufflehog`) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add `.env` to `.gitignore`.

#136 AS-002 -- SECRET SCAN

HIGH

OWASP: AA-05 | MITRE: T1552 | STRIDE: Information Disclosure

Hardcoded secrets, API keys, tokens, or credentials in source code

[tests/test_research_polling.py:12](#)

Potential secret: possible hardcoded secret

```
10 |     cookies = {"SID": "test_sid"}  
11 |     with patch.object(NotebookLMClient, "_refresh_auth_tokens"):  
> 12 |         client = NotebookLMClient(cookies=cookies, csrf_token="old_token", session_id="old_sid")  
13 |         return client  
14 |
```

Remediation:

1. Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv). 2. Add pre-commit hooks (`detect-secrets`, `trufflehog`) to block secret commits. 3. Rotate any exposed keys immediately. 4. Add `.env` to `.gitignore`.

#137 AS-004 -- PROMPT INJECTION

HIGH

OWASP: AA-02 | MITRE: T1190 | STRIDE: Tampering

File input flows near prompt or instruction handling create injection vectors

[tests/test_wsl_firewall_scope.py:41](#)

File input near prompt/instruction handling — injection vector

```
39 |  
40 |  
> 41 | def test_privilege_fallback_command_is_not_wide_open(_wsl_env, monkeypatch):  
42 |     monkeypatch.setattr(  
43 |         wsl.subprocess, "run", Mock(return_value=_run_result(1, "Access is denied"))
```

Remediation:

1. Sanitize all file contents before injecting into prompts (strip control chars, limit length). 2. Use structured input schemas — never concatenate raw file content into system prompts. 3. Implement input/output firewalls (Rebuff, LLM Guard). 4. Separate data plane from control plane in prompt construction.

Remediation Checklist

Prioritized by severity. Address CRITICAL items first.

Quick Win

`npm install mcp-secure`

Adds cryptographic passports, signed messages, tool verification, and audit logging.

CRITICAL (2)

- ☐ **AS-001: unsafe execution**
Replace `exec()/eval()` with AST-parsed execution (`ast.literal_eval` for data, `RestrictedPython` for code).
- ☐ **AS-005: injection pattern**
Validate and sanitize all inputs with strict allowlists.

HIGH (5)

- ☐ **AS-008: excessive agency**
Implement mandatory human-in-the-loop for destructive operations (`delete`, `write`, `execute`, `send`).
- ☐ **AS-006: sandboxing**
Run all agent-generated code in Docker containers with `--network=none` and read-only filesystem.
- ☐ **AS-004: prompt injection**
Sanitize all file contents before injecting into prompts (strip control chars, limit length).
- ☐ **AS-011: data exfiltration**
Implement URL allowlisting — agents can only make HTTP requests to pre-approved domains.
- ☐ **AS-002: secret scan**
Move all secrets to environment variables or a vault (HashiCorp Vault, AWS Secrets Manager, dotenv).

Methodology & Standards

Scanner	mcps-audit v1.0.0
Rules Version	1.1.0
Scanned At	18/09/2026, 14:03:49
Report ID	MCPS-MU6M4B8D

Referenced Standards

- OWASP MCP Top 10**
Security risks for Model Context Protocol servers and clients.
owasp.org/www-project-mcp-top-10
- OWASP Agentic AI Top 10**
Security risks for autonomous AI agents.
owasp.org/www-project-agentic-ai-top-10
- MCPS -- IETF Internet-Draft**
Cryptographic security layer for MCP: draft-sharif-mcps-secure-mcp.
datatracker.ietf.org/doc/draft-sharif-mcps-secure-mcp
- MITRE ATT&CK**
Adversary tactics and techniques knowledge base.
attack.mitre.org
- STRIDE Threat Model**
Microsoft threat classification framework.
microsoft.com/en-us/security/blog/stride

AgentSign
Cryptographic identity and security for AI agents
agentsign.dev